

# Cyber Security

## Lab 3: Finger printing and Scanning





# Table of Contents

- **Footprinting and Scanning**
  - ❑ 3.1 Disclaimer
  - ❑ 3.2 Mapping a Network
  - ❑ 3.3 Port Scanning



# Mapping A network



# Mapping A Network

- After collecting information about the target organization during the information gathering stage, a penetration tester proceeds to the fingerprinting and enumeration of the nodes running on the client's network; this is the infrastructure part of information gathering.
- The techniques you are going to see work both on local and remote networks.
- As you know, every host connected to the Internet or a private network must have a unique IP address which identifies it.
- How can a penetration tester determine what hosts, sitting in an in-scope network, are up and running



# Why Map a (Remote) Network?

A company asks for a penetration test, and the following address block is considered in scope: 200.200.0.0/16.

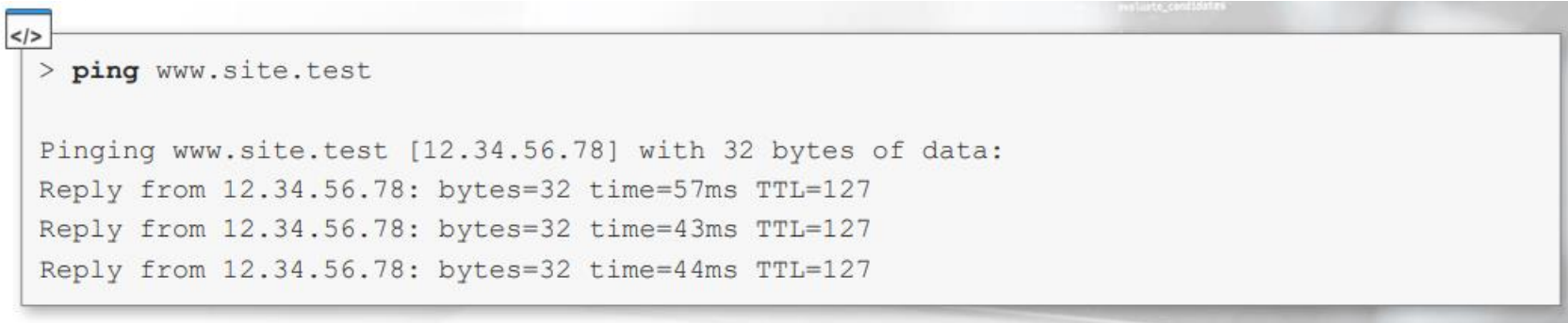
A sixteen-bit long netmask means the network could contain up to 2<sup>16</sup> (65536) hosts with IP addresses in the 200.200.0.0 - 200.200.255.255 range.

The penetration tester needs a way to find which of the 65536 IP addresses are **assigned to a node**.



# Ping Sweeping

You probably already know the ping command; it is a utility designed to test if a machine is alive on the network. You can run a ping in every major operating system by using the command line:

A terminal window with a title bar that includes a code icon and the text 'evaluate\_candidates'. The terminal shows a command prompt followed by the 'ping' command for 'www.site.test'. The output shows three successful replies from the IP address 12.34.56.78 with varying response times and a TTL of 127.

```
</>  
> ping www.site.test  
  
Pinging www.site.test [12.34.56.78] with 32 bytes of data:  
Reply from 12.34.56.78: bytes=32 time=57ms TTL=127  
Reply from 12.34.56.78: bytes=32 time=43ms TTL=127  
Reply from 12.34.56.78: bytes=32 time=44ms TTL=127
```



# fping

Fping is a Linux tool which is an improved version of the standard ping utility. You can use Fping to perform ping sweeps.

It is installed by default on Kali Linux, and you can run it from the command line using the following syntax:

## Example:

```
# fping -a -g IPRANGE
```



# fping

The `-a` option forces the tool to show only alive hosts, while the `-g` option tells the tool that we want to perform a ping sweep instead of a standard ping.

You can define an IP range by using the CIDR notation or by specifying the start and the ending addresses of the sweep.

## Examples:

```
</> # fping -a -g 10.54.12.0/24
# fping -a -g 10.54.12.0 10.54.12.255
```





# fping

When running Fping on a LAN you are directly attached to, even if you use the -a option, you will get some warning messages (ICMP Host Unreachable) about offline hosts.

To suppress those messages, you can redirect the process standard error to /dev/null.

## Example:

```
</> # fping -a -g 192.168.82.0 192.168.82.255 2>/dev/null  
192.168.82.1  
192.168.82.11  
192.168.82.112  
192.168.82.171  
192.168.82.202
```



# NMAP

You can perform a ping scan by using the `-sn` command line switch. You can specify your targets on the command line in CIDR format as a range and by using wildcard notation.

## **Examples:**



```
# nmap -sn 200.200.0.0/16  
# nmap -sn 200.200.123.1-12  
# nmap -sn 172.16.12.*  
# nmap -sn 200.200.12-13.*
```



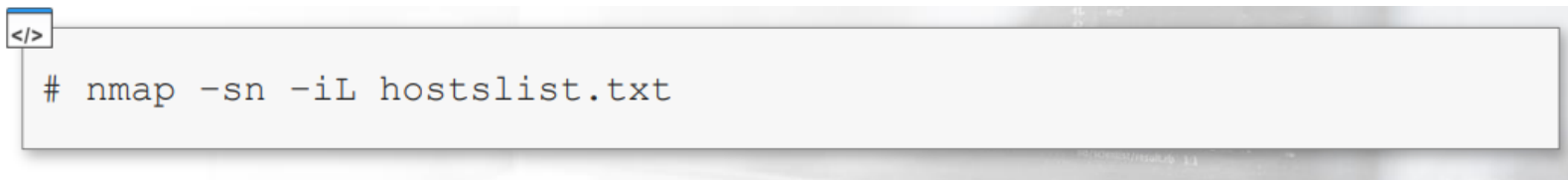
# NMAP

Moreover, you can save your host list in a file and use the input list `-iL` command line switch.

You can save your host list and invoke it with Nmap by using the input command line switch in conjunction with `-sn`.

This way you will conduct a ping scan against each host that is in the given list. Please note, that in this example we are naming our saved file `hostslist.txt`

## **Example:**



```
</>
# nmap -sn -iL hostslist.txt
```

A terminal window with a light gray background and a dark gray border. The window has a small icon in the top-left corner. The command `# nmap -sn -iL hostslist.txt` is entered in the terminal.

# OS Fingerprinting

Once you have mapped the network, you end up with a list of live hosts. These are the hosts that responded to pings or to nmap probes, but you still don't know what software the hosts are running and what role they have in the network.

Are they routers, servers or clients? At this stage, you can only say that they are up and running hosts connected to the network.

- **OS fingerprinting** is the process of determining the operating system used by a host on a network.



# OS Fingerprinting

To fingerprint an operating system you have to send network requests to the host and then analyze the responses you get back.

This is possible because of some tiny differences in the network stack implementation of the various operating systems.

Finally, the signature is compared against a database of known operating systems signatures.

This process exploits differences in the network stack implementation, so there is some guesswork; but, using the best tools and supporting them with experience, you can achieve prettyOS Fingerprinting with Nmap reliable results!



# OS Fingerprinting with NMAP

To perform OS fingerprinting with nmap, you have to use the `-O` command line option and specify your target(s).

You can also add the `-Pn` switch to skip the ping scan if you already know that the targets are alive.



```
# nmap -Pn -O <target(s)>
```



# OS Fingerprinting with NMAP

You can fine-tune the OS fingerprinting process by using the following options:



## OS DETECTION:

- O: Enable OS detection
- osscan-limit: Limit OS detection to promising targets
- osscan-guess: Guess OS more aggressively

Choosing a lighter or more aggressive OS detection depends on the engagement.



# OS Fingerprinting with NMAP

If you really need to detect the OS of a machine you know is alive, but is not responding to ping probes, you could run:

```
</> # nmap -Pn -O <target>
```

On the other hand, if you have to scan thousands of hosts you could at first limit OS reconnaissance to just the promising ones.

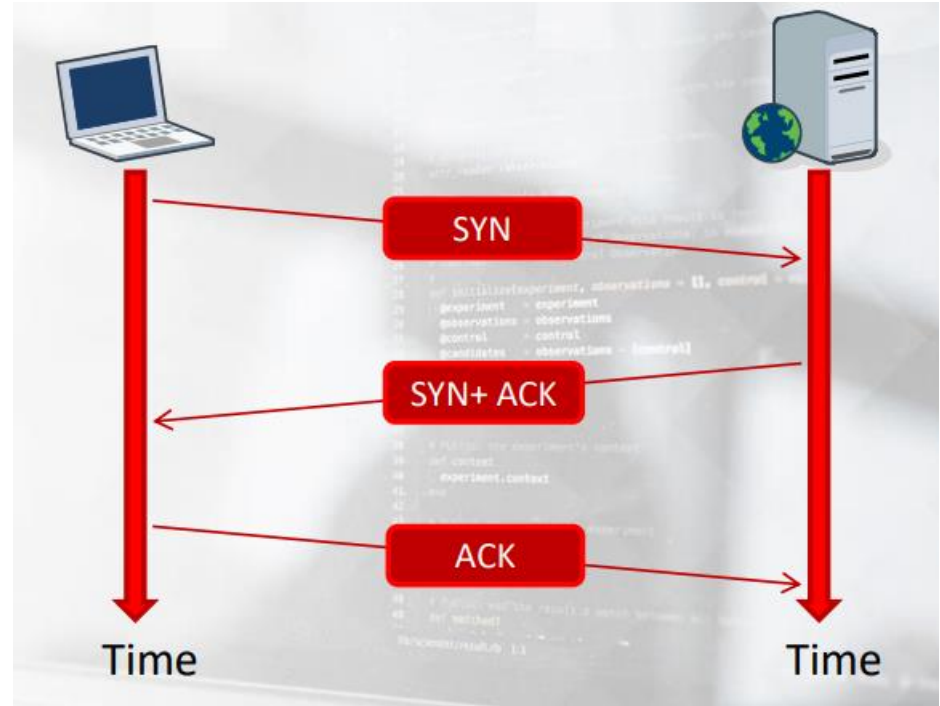
```
</> # nmap -O --osscan-limit <targets>
```





# Port Scanning

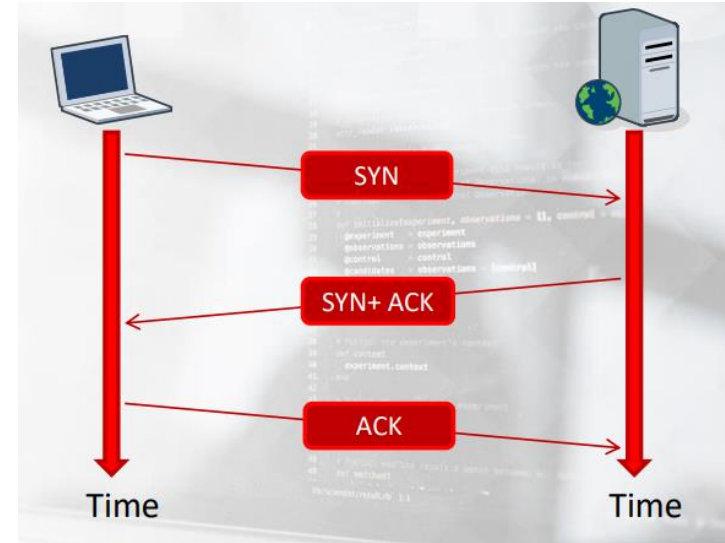
- Port scanning is a process used to determine what TCP and UDP ports are open on target hosts. Moreover, it lets you know which daemon, in terms of software and version, is listening on a specific port.
- Using this technique, you can create a list of potential weaknesses and vulnerabilities to check during the following vulnerability assessment phase.
- As you can recall from the Networking module, a daemon is a piece of software running on a server to provide a given service. A daemon also listens on a specific port.
- The ultimate goal of port scanning/service detection is to find the software name and version of the daemons running on each host.



# TCP Three Way Handshake

## TCP Handshake:

- When a client wants to connect to a server, it first sends a packet with the SYN flag enabled.
- The server then responds by sending a packet with both SYN and ACK flags enabled.
- Finally, the client replies back by sending a packet with the ACK flag enabled and the actual data transmission can start.

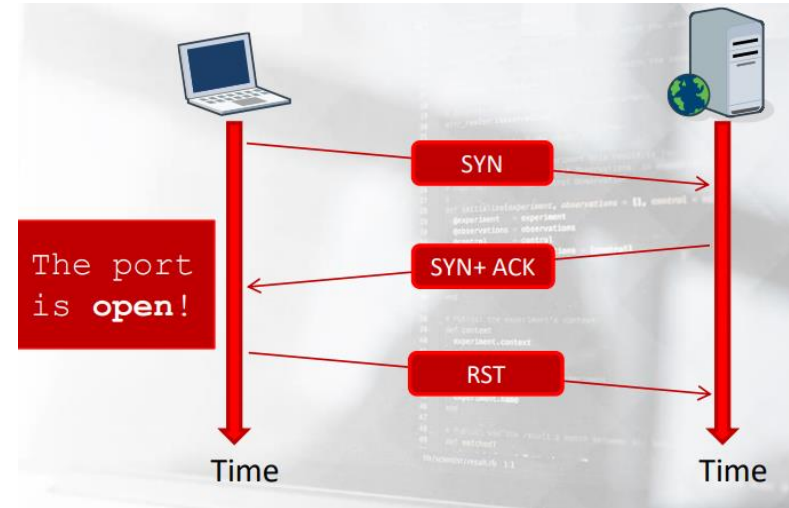


# TCP SYN Scan

## TCP Handshake:

The simplest way to perform a port scan is trying to connect to every port.

- If the scanner receives a RST packet, then the port is closed.
- If the scanner can complete the 3-way handshake, then the port is open. After connecting, the scanner sends an RST packet to the target host to abruptly close the connection



# Scanning with NMAP

To pass a scan type or an option on the command line, you have to use a dash "-" followed by one or more letters to specify your options. In the following slides, you will see some common command line switches you can use to perform different scan activities.



```
# nmap -sS -sV -O --osscan-limit 192.168.1.0/24
```

## The most used scan types are:

- sT performs a TCP connect scan
- sS performs a SYN scan
- sV performs a version detection scan

While TCP connect scans and SYN scans works as you saw earlier, the version detection scan does something more.



# TCP Connect Scan with Nmap

To perform a TCP connect scan, you can use the command line switch `-sT`.

Keep in mind that this type of scan gets recorded in the application logs on the target systems, as every daemon will receive a connection from the scanning machine.



```
# nmap -sT <target>
```



# TCP SYN Scan with Nmap

To perform a TCP SYN scan, you can use the command line switch `-sS`. This type of scan is also known as stealth scan because there is no (full) connection to the target daemons. Note that a well-configured IDS will still detect the scan!



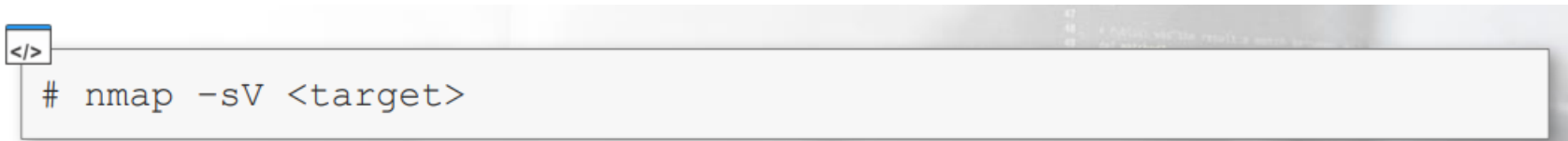
```
# nmap -sS <target>
```



# Version Detection Scan with Nmap

To perform a version detection scan, you can use the command line switch `-sV`.

This type of scan mixes a TCP connect scan with some probes, which are used to detect what application is listening on a particular port; this is not stealthy but very useful.

A terminal window with a light gray background. The command `# nmap -sV <target>` is entered in a dark gray input field. A small icon with `</>` is visible in the top left corner of the terminal window.

```
# nmap -sV <target>
```

